Natural Language Processing: Assignment 3

Efe Özbal (s5996821), Sigurdur Haukur Birgisson (s5982960), Josh van Vliet (s5950465)

April 6, 2026

1 Dataset & Setup

To classify news articles into one of four categories (world, sports, business, and sci/tech), we use the AG News dataset from Hugging Face [Zhang et al., 2015]. This dataset consists of news articles with associated labels, titles, and descriptions. Each class contains 30,000 training samples and 1,900 testing samples, resulting in a total of 120,000 training samples and 7,600 testing samples.

We used the official Hugging Face split (120,000 train and 7,600 test). The training set was split into a new training set (of size 108,000) and a development set (of size 12,000) using stratified sampling with a fixed random seed of 67 to ensure reproducibility. The final split sizes are: train 108,000, dev 12,000, and test 7,600.

Preprocessing. We combined the title and description of each news article into a single text input. We cleaned the text by HTML unescaping to convert HTML entities back to their original characters (like `#39;` to `'`) and removed any remaining HTML tags. We also unescaped LaTeXentities (\\ etc.), collapsed backslashes and normalized whitespace. We converted the labels from 1-based to 0-based indexing (i.e., world: 0, sports: 1, business: 2, sci/tech: 3) for easier handling in the classification models.

Tokenization. We used BERT Tokenizer, which is based on WordPiece tokenization, to tokenize the text inputs. Similar to BPE, WordPiece is a subword tokenization method that can effectively handle out-of-vocabulary words (OOV) and reduce the vocabulary size while still capturing meaningful subword units. We used the pre-trained BERT tokenizer ($bert-base-uncased$) from Hugging Face, which has a vocabulary size of 30,000 tokens. Both BERT and LSTM models were trained using the same tokenizer to ensure consistency in input representation.

2 Models

In this assignment, we compare two neural models for news classification: a Convolutional Neural Network (CNN) and a Long Short-Term Memory network (LSTM). To make the comparison fair, both models use the same data splits, preprocessing pipeline, and evaluation setup. The overall classification task pipelines is illustrated in Figure **??**.

Shared input representation. Both models receive the same preprocessed and tokenized text (see Section 1 for preprocessing and tokenization details). Sequences are padded or truncated to the chosen maximum length. Both models begin with a trainable embedding layer that maps token ids to dense vectors. Note that the CNN has inherent feature extraction through its convolutional layers, while the LSTM relies on the embeddings and recurrent connections for representation learning.

LSTM model. The LSTM model is based on the Long Short-Term Memory architecture introduced by [Hochreiter and Schmidhuber, 1997]. It reads the input sequence token by token, which makes it suitable for modelling word order and longer dependencies in the text. We use a stacked 2-layer LSTM. To avoid learning from padding tokens, we handle variable-length sequences by skipping over padded positions during the recurrent computation, so the model only processes the real tokens in each example. For classification, we use the final hidden state of the top LSTM layer as the representation of the whole article. In the bidirectional setting, the model combines information from both directions before making the final prediction. Dropout is applied before the final linear layer. This model architecture was chosen as it was the best performing model in Assignment 2; however, due to the change in tokenization for a fair comparison, we re-tuned the dropout rate and learning rate to ensure a fair comparison with the BERT model. As training the LSTM is not the focus of this assignment, no further details on the training process will be provided, but hyperparameters used for final LSTM training can be seen in Table 1.

BERT model. The BERT model is based on the pre-trained Bidirectional Encoder Representations from Transformers (BERT) architecture [Devlin et al., 2019]. We use the $bert-base-uncased$ variant, which has 12 layers, a hidden size of 768, and 12 attention heads, resulting in a total of 110 million parameters. The model is fine-tuned on our classification task by adding a feed-forward neural network classification head on top of the [CLS] token representation. The BERT model is trained end-to-end, meaning that the weights of the pre-trained BERT layers and the classification head are updated during training on our dataset. Before using the regular learning rate for fine-tuning, we apply a warmup phase of 500 steps where the learning rate gradually increases from zero to the specified learning rate. This helps stabilize training and prevents large updates to the pre-trained weights in the early stages of fine-tuning. As in LSTM, dropout is applied at the classification head to prevent overfitting.

Hyperparameters. Table 1 shows the hyperparameter search space for BERT model and the best hyperparameters for LSTM from Assignment 2. The best setting for BERT was selected based on macro-F1 on the development set.

Parameter	**BERT**	**LSTM**
Embedding dim d	—	256
Hidden dim H	768	256
Sequence length	128	128
Learning rate	2e-5	3e-4
Weight decay	0.01	0.01
Dropout	—	0.4
Layers L	12	2
Bidirectional	True	True
Attention heads A	12	—
Head dim	256	—
Head dropout	0.1	—

Table 1: Hyperparameter search space for the BERT and best hyperparameters for the LSTM model.

Shared training settings. Both models were trained with the Adam optimiser. We used gradient clipping with a maximum norm of 1.0 and early stopping with a patience of 2 epochs based on validation loss. The batch size for BERT was set to 16, while for the LSTM it was set to 32 to boost the training times.

Robustness/Slicing: Keyword Masking Probe. As the first robustness/slicing study for BERT model, we implemented a simple keyword masking probe to test how much the models rely on specific keywords for classification. We identified the top 15 most informative keywords for each class based on the training data

using TF-IDF scores. Then, we created a modified test set where these keywords were masked out (replaced with a special [MASK] token). We evaluated the models on this masked test set to see how much their performance drops compared to the original test set. A significant drop in performance would suggest that the model relies heavily on these keywords for classification, while a smaller drop would indicate that the model is using more contextual information and is less sensitive to specific keywords. This analysis helps us understand the robustness of the models to changes in key lexical cues and whether they are learning more generalizable patterns or just memorizing specific terms.

Robustness/Slicing: Label-Noise Sensitivity (Simple). As the second robustness/slicing study for BERT model, we implemented a simple label-noise sensitivity analysis. We reduce the training set sizes to 25%, 50%, and 75% of the original training set, and evaluate the models trained on these smaller subsets on the full test set. This allows us to see how sensitive the models are to the amount of training data, their learning efficiency, and generalization capabilities. If the model's performance drops significantly with less training data, it may indicate that the model is heavily reliant on large amounts of data to learn effectively. Conversely, if the performance remains relatively stable even with reduced training data, it suggests that the model is able to learn useful representations and generalize well even with limited data.

3 Experimental Protocol

For BERT model, we first performed hyperparameter tuning on the training set. BERT was tuned using grid search over the parameter ranges shown in Table 1. Training each configuration with early stopping based on validation loss, we selected the best hyperparameters based on macro-F1 on the development set.

After selecting the best hyperparameters for a given model, we trained that model again on the train data and evaluated it on the test set. The final training runs used the Adam optimiser, a batch size of 512, gradient clipping with a maximum norm of 1.0, and a maximum of 15 epochs. Unlike the tuning stage, the final training procedure did not apply early stopping; instead, models were trained for the full configured number of epochs and then evaluated on the test set. We made this choice to obtain directly comparable learning curves across all runs and to analyse convergence behaviour in more detail. However, this design introduces a tradeoff: by prioritising diagnostic information, we may give up some generalisation performance relative to selecting the best validation checkpoint with early stopping. The final test scores should therefore be interpreted as results from a fixed-length diagnostic training protocol, not necessarily as the best achievable numbers for each architecture.

We report accuracy and macro-F1 as our main evaluation metrics, with macro-F1 used as the primary measure for model selection because it gives equal weight to all four classes. In addition, we generated classification reports, confusion matrices, learning curves, and files with misclassified test examples for later error analysis. The test set was only used for final evaluation after model selection.

As our ablation study, we varied the maximum input sequence length while keeping the rest of the hyperparameters fixed to the respective base models' constants (same setup). We ran each model with maximum sequence lengths of 64, 128, and 256 tokens, resulting in six main experiment settings: CNN-seq64, CNN-seq128, CNN-seq256, LSTM-seq64, LSTM-seq128, and LSTM-seq256. This design allows us to compare the two architectures directly and also study how much performance depends on the amount of input context available to the model.

4 Results

4.1 Best hyperparameters

Table 2 summarises the best hyperparameters for BERT model.

Parameter	**Value**
Hidden dim H	768
Sequence length	128
Learning rate	2e-5
Weight decay	0.01
Layers L	12
Bidirectional	True
Attention heads A	12
Head dim	256
Head dropout	0.1

Table 2: Best hyperparameters for the BERT model.

4.2 Test set performance of base models

Figure 1 shows the distribution of dev macro-F1 scores for both models across all hyperparameter search trials. The distributions are relatively tight, especially for the LSTM, indicating that the tuning process was stable and not overly sensitive to hyperparameter choices. The CNN shows a slightly wider distribution, suggesting that its performance may be more sensitive to hyperparameter settings than the LSTM. Overall, both models achieved high dev macro-F1 scores across a range of hyperparameters, which is a positive sign for their generalization capabilities.

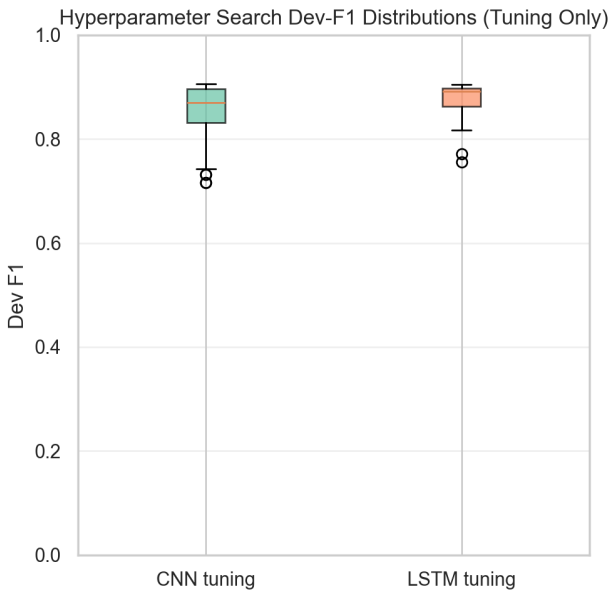

Figure 1: Distribution of dev macro-F1 scores across all hyperparameter search trials, grouped by model. Higher and tighter distributions indicate more stable tuning.

Table 3 reports tokenizer statistics for the shared BPE vocabulary used across all runs. Because BPE decomposes unknown words into subword units rather than discarding them, coverage is effectively complete, but complete coverage does not imply zero fragmentation. Words are split into roughly 1.19 subword tokens on average, meaning the tokenizer is unlikely to hurt performance through out-of-vocabulary terms, though named entities and domain-specific vocabulary may still be harder to represent consistently when broken across multiple pieces.

Split	**Docs**	**Types**	**Tokens**	**Words**	**Tokens/Word**
Train	108000	19565	4859372	4098055	1.1858
Dev	12000	18698	544016	457324	1.1896
Test	7600	17987	341726	287540	1.1884
Test split by length bucket					
Bucket	**Docs**	**Doc Share (%)**	**Tokens**	**Token Share (%)**	**Tokens/Doc**
Short	154	2.0	3595	1.1	23.3
Medium	6941	91.3	300830	88.0	43.3
Long	505	6.6	37301	10.9	73.9

Table 3: Tokenizer coverage summary for the shared BPE tokenizer used in all runs. OOV metrics are omitted. Tokens/Word indicates average subword split rate. (Tokenizer: BPETokenizer, vocab size = 19980.)

The test results are shown in Table 4. These numbers come from the fixed-length final training protocol, where early stopping was intentionally disabled to obtain complete learning curves. They are therefore useful for comparing model behaviour under the same schedule, but they should not be read as guaranteed best-checkpoint performance for either architecture. At the main comparison sequence length of 128 tokens, the CNN slightly outperformed the LSTM, achieving a test accuracy of 90.71% and a macro-F1 of 90.69%, compared to the LSTM's accuracy of 90.57% and macro-F1 of 90.59%.

Interestingly, the ablation results reveal that longer sequences do not consistently improve performance. For the CNN, the best test macro-F1 was actually achieved at sequence length 64 (90.96%), while the highest dev F1 was at 256 (91.09%). The LSTM showed a more monotonic trend, improving from 90.32% at seq64 to 90.74% at seq256. The gap between the two models was relatively small across all sequence lengths, with a difference of at most 0.64 percentage points in macro-F1 (at seq64).

Model	**Seq Len**	**Dev F1**	**Test Acc**	**Macro F1**
Main model comparison (seq length = 128)				
CNN	**128**	0.9101	0.9071	0.9069
LSTM	**128**	0.9090	0.9057	0.9059
Ablation: sequence length				
CNN	64	0.9096	**0.9099**	**0.9096**
CNN	256	**0.9109**	0.9075	0.9072
LSTM	64	0.9086	0.9034	0.9032
LSTM	256	0.9076	0.9074	0.9074

Table 4: Main comparison at sequence length 128 (CNN vs. LSTM), followed by sequence-length ablations. Dev F1 is the best development macro-F1 found during tuning. Best scores per metric column are bolded.

4.3 Ablation study: effect of maximum sequence length

Figure 2 shows how test accuracy and macro-F1 vary with sequence length.

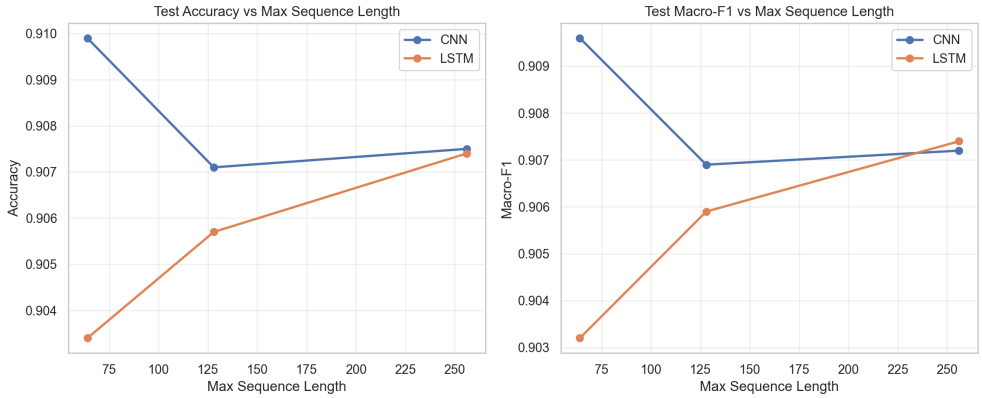

Figure 2: Test accuracy and macro-F1 as a function of maximum sequence length for CNN and LSTM.

The overall trend is clear: The CNN does not benefit from longer sequences, with the best test macro-F1 at seq64. The LSTM, on the other hand, shows a consistent improvement with longer sequences, achieving its best test macro-F1 at seq256. This suggests that the LSTM is better able to leverage the additional context provided by longer input sequences, while the CNN may be more prone to overfitting or may not effectively utilize the extra information. This is especially evident by the fact that the CNN outperforms the LSTM at seq64, but is outperformed by the LSTM at seq256 (see Figure 2).

4.4 Class-wise analysis

Figure 3 presents the per-class macro-F1 scores. The confusion matrices per model and sequence length are shown in Appendix B along with the full classification reports.

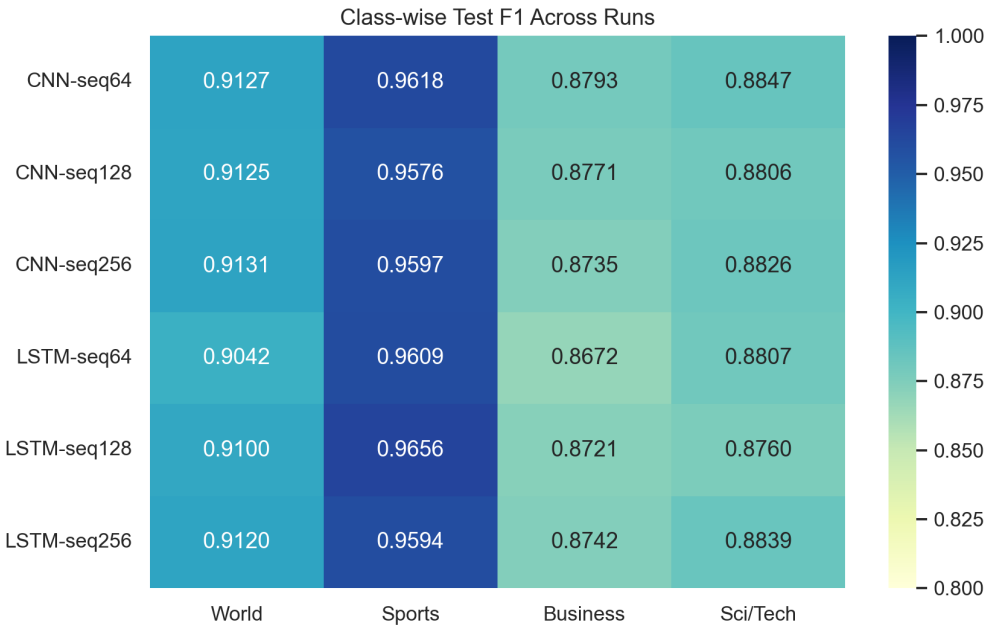

Figure 3: Heatmap of per-class test F1 scores across all runs. World and Sports are easier to classify; Business is the most challenging.

The *World* and *Sports* classes are generally easier to classify, with F1 scores above 90% in most configurations. The *Sci/Tech* class is slightly harder, with F1 scores typically around 87–88%. The most challenging class is *Business*, where F1 scores range from 86% to 87%, depending on the model and sequence length. One detail that is not fully visible from the heatmap alone is that the LSTM at seq128 has noticeably lower recall on the *World* class than the CNN at the same sequence length (89.63% vs. 92.16%). This suggests that, even when the overall macro-F1 values are very close, the CNN is somewhat better at recovering *World* examples at the main comparison length. A plausible explanation is that *World* articles often contain relatively local lexical cues, such as country names, political actors, and event words, which may be captured more reliably by the CNN.

4.5 Learning curves

Figure 4 displays the learning curves for the final training runs on the training set and validation set for all six runs (CNN and LSTM at seq64, seq128, and seq256) with the tuned hyperparameters from the dedicated tuning runs. Both training and validation curves are shown for loss and accuracy. The F1 curves are shown in Appendix C.

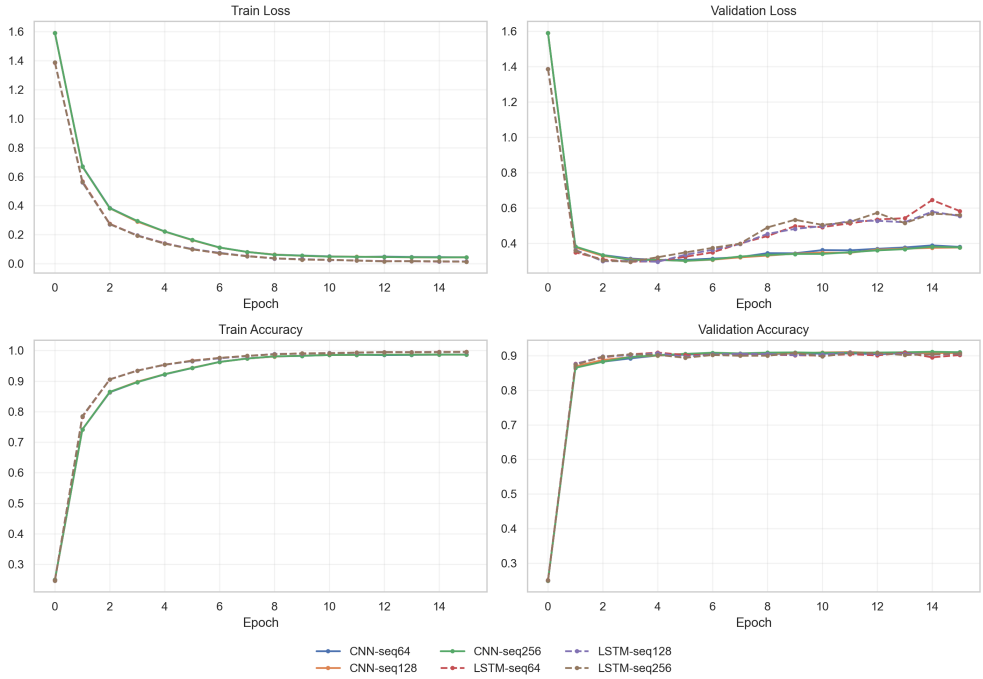

Figure 4: Training and validation loss/accuracy curves for all six runs. Both models show rapid early convergence and stable learning trajectories.

Both models show rapid convergence, with validation loss decreasing sharply in the first few epochs and then stabilizing. Both models converge to similar final validation accuracies at a similar number of epochs (e.g., around 1–2 epochs). The CNN shows a slightly more stable validation loss curve, while the LSTM shows signs of overfitting after epoch 4, where the validation loss starts to increase while training loss continues to decrease. Most notably, the LSTM at seq256 converges more rapidly as indicated by the steepest initial drop in validation loss, which may explain its superior performance at this sequence length. At the same time, these curves also illustrate the cost of disabling early stopping in the final runs: some models appear to move past their best validation region, so the reported test numbers should be read with that caveat in mind.

4.6 Summary

Under the fixed-length final training protocol, the best overall test result is achieved by the CNN at the shortest sequence length (seq64), with a macro-F1 of 90.96%. The LSTM reaches its best result at seq256 (90.74%), indicating that it benefits more from longer input sequences, while the CNN performs best with shorter inputs. Both models show stable tuning and learning trajectories, with the Business class being the most challenging to classify.

5 Error Analysis

To better understand the remaining errors, we examined the first 20 misclassified test examples saved for each of the six runs. Rather than looking at isolated mistakes, we focused on examples that reappeared across multiple models and sequence lengths. This makes it possible to identify systematic weaknesses instead of noise from a single run. Full listings of all misclassified examples are provided in Appendix A.

The clearest pattern is confusion between *Sci/Tech* and *Business*. Across all six runs, *Sci/Tech* articles misclassified as *Business* is the single most common error type. This is unsurprising: many technology articles in AG News employ business language such as *market*, *shares*, *workers*, or company names. Examples that recur persistently across nearly every run include "IBM to hire even more new workers" (App. **??**, Ex. 6; App. **??**, Ex. 5; App. **??**, Ex. 4), "Bill Clinton Helps Launch Search Engine" (App. **??**, Ex. 20; App. **??**, Ex. 20; App. **??**, Ex. 17; App. **??**, Ex. 19), and "Intuit Posts Wider Loss After Charge" (App. **??**, Ex. 17; App. **??**, Ex. 16; App. **??**, Ex. 14; App. **??**, Ex. 15; App. **??**, Ex. 17). The reverse confusion also occurs: Business articles mislabeled as Sci/Tech include "Intel to delay product aimed for high-definition TVs" (App. **??**, Ex. 10; App. **??**, Ex. 9; App. **??**, Ex. 9; App. **??**, Ex. 9; App. **??**, Ex. 6; App. **??**, Ex. 7) and "Yahoo! Ups Ante for Small Businesses" (App. **??**, Ex. 11; App. **??**, Ex. 10; App. **??**, Ex. 10; App. **??**, Ex. 11; App. **??**, Ex. 7; App. **??**, Ex. 8), both of which are misclassified in every single run. These systematic errors reveal that the boundary between technological innovation and economic reporting is inherently ambiguous in this dataset.

Another recurring pattern is confusion between *World* and *Business*. Articles about politics or international affairs are repeatedly misclassified as *Business* when they mention oil prices, trade, or financial framing. "Oil prices bubble to record high" appears in all six runs (App. **??**, Ex. 12; App. **??**, Ex. 11; App. **??**, Ex. 11; App. **??**, Ex. 12; App. **??**, Ex. 8; App. **??**, Ex. 9), as does "Venezuela Prepares for Chavez Recall Vote" (App. **??**, Ex. 8; App. **??**, Ex. 6; App. **??**, Ex. 6; App. **??**, Ex. 5). The article "Future Doctors, Crossing Borders" (App. **??**, Ex. 13; App. **??**, Ex. 12; App. **??**, Ex. 13; App. **??**, Ex. 16; App. **??**, Ex. 12; App. **??**, Ex. 12) is also misclassified in every run, which is a particularly interesting case since its content sits between health and education without any obvious economic keywords. These mistakes suggest that both models rely heavily on common economic keywords to identify the Business class, which leads to confusion when those keywords appear in political or international news contexts.

We also observe a consistent confusion between *World* and *Sports*, particularly around Olympics-related articles. "Live: Olympics day four" is misclassified as *Sports* in runs 1, 2, and 3 (App. **??**, Ex. 9; App. **??**, Ex. 8; App. **??**, Ex. 8), despite being labelled *World*. Similarly, "Afghan women make brief Olympic debut" (App. **??**, Ex. 14; App. **??**, Ex. 13) and "Security scare as intruder dives in" (App. **??**, Ex. 15; App. **??**, Ex. 10; App. **??**, Ex. 10) are repeatedly predicted as the wrong class. These cases suggest that Olympic coverage, which blends international news framing with sporting events, is difficult for both architectures. Conversely, some Sports articles such as "Hamilton Wins Cycling Time Trial Event" (App. **??**, Ex. 17; App. **??**, Ex. 19) are misclassified as *World*, likely because the tour de France context and the athlete's name carry little explicit sporting vocabulary.

Some errors are caused by lexical ambiguity in short texts. "E-mail scam targets police chief" (App. **??**, Ex. 2; App. **??**, Ex. 2; App. **??**, Ex. 2; App. **??**, Ex. 2) is repeatedly misclassified as *World*, most likely because

words such as *police* and *chief* are strongly associated with political or crime-related world news. Similarly, "Super ant colony hits Australia" (App. **??**, Ex. 4; App. **??**, Ex. 3; App. **??**, Ex. 4) is consistently predicted as *World*, despite belonging to *Sci/Tech*: the country name and event-style headline likely dominate over any scientific signal.

Comparing the two models, the overall error types are strikingly similar. The LSTM shares virtually all of the same repeat errors as the CNN, especially the *Sci/Tech–Business* confusions, which suggests that additional sequential modelling capacity is not sufficient to resolve these cases. This is consistent with the Results section: even though the CNN performs best overall at seq64 and the LSTM improves at seq256, both models still fail on many of the same boundary cases. Notably, "Selling Houston Warts and All, Especially Warts" (App. **??**, Ex. 19; App. **??**, Ex. 19; App. **??**, Ex. 16; App. **??**, Ex. 18; App. **??**, Ex. 20) is misclassified in almost every run, and interestingly by different classes depending on the model and sequence length (*World* for CNN, *Sci/Tech* for LSTM), which shows that some examples are just ambiguous, rather than any modelling weakness.

Compared with Assignment 1 (TF-IDF with Linear SVM and Logistic Regression), the overall error profile is very similar. In both assignments, the dominant confusion is still *Sci/Tech* vs. *Business*, with *World* vs. *Business* as another recurring boundary. This consistency across the modeling techniques suggests that the main source of error is label ambiguity in the dataset. Many headlines combine technological and financial language, or mix political and economic framing. Moving from sparse TF-IDF features to neural sequence models improves scores, but does not remove these fundamental ambiguities.

Overall, the error analysis suggests that the main challenge is separating articles near the boundary between related topics. The distinction between *Sci/Tech* and *Business* remains the hardest for both models, consistent with the class-wise results in Section 4 where these two categories had the lowest per-class F1 scores.

6 Discussion

Our results suggest that this task depends a lot on short local patterns in the text. That helps explain why the CNN is already strong at sequence length 64: even with shorter inputs, it can pick up useful topic phrases and keywords. So the CNN advantage at short length is not only a score difference, but also a good match between the model and the kind of signal in AG News.

At the same time, the BiLSTM improves more as we increase sequence length, and at 256 tokens it slightly surpasses the CNN (Table 4; 90.74% vs 90.72% macro-F1). A likely reason is that the extra context helps the LSTM in examples where early words are not enough to decide the class. This seems especially relevant for Business and Sci/Tech, where articles often contain mixed vocabulary. Even so, the best overall configuration is still CNN-seq64, which suggests that local cues remain very important for this dataset.

The class-wise results support this broader picture. World and Sports are generally easier for both models, while Business and Sci/Tech remain harder. This tells us that the remaining errors are mostly about separating semantically close topics, not about basic model failure.

Overall, sequence length affects the two architectures differently. For the CNN, increasing sequence length does not reliably improve performance, and the best test result is already reached at seq64. For the LSTM, in contrast, performance increases monotonically from seq64 to seq256, which suggests that this architecture does make better use of additional context. This model-specific contrast indicates that the two architectures exploit context differently: the CNN appears to benefit most from short, highly informative local patterns, whereas the LSTM gains more steadily from longer inputs. However, the remaining errors in both models are still very similar, which indicates that the main limitation is not sequence length alone, but the difficulty of separating overlapping news categories such as Business and Sci/Tech.

7 Limitations

This study has a few limitations. First, we only evaluate on AG News, so our conclusions may not transfer to datasets with different writing styles or label structures. A second limitation concerns tokenizer analysis: although the shared BPE tokenizer (vocab size 19,980, max vocab size 20,000) shows full coverage with zero OOV and a stable split rate of about 1.19 tokens per word, these coverage statistics do not guarantee that subword segmentation preserves all task-relevant distinctions. In particular, named entities and domain-specific terms may still be split into pieces that are harder for the models to use consistently.

Another important limitation is the tuning setup. Hyperparameter search was done with only 5 epochs per configuration, and we did not explicitly verify that this was long enough for strong and weak configurations to separate clearly. This means some settings may have looked worse simply because they learned more slowly, not because they were truly worse.

Regularisation is another place where our choices may have been too limited. In the best configurations, weight decay was either very small or zero, and the learning curves suggest that the LSTMs overfit slightly more than the CNNs. Because of this, it would have been useful to test higher weight decay values more aggressively, especially for the LSTM models.

Finally, we compare only CNN and LSTM models trained from scratch, so this is a comparison between two classical neural architectures rather than a comparison with the current strongest text-classification models. That does not affect the internal comparison in this report, but it does limit how far the absolute performance numbers can be taken.

References

[Devlin et al., 2019] Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). Bert: Pre-training of deep bidirectional transformers for language understanding.

[Hochreiter and Schmidhuber, 1997] Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8):1735–1780.

[Zhang et al., 2015] Zhang, X., Zhao, J., and LeCun, Y. (2015). Character-level convolutional networks for text classification. In *Advances in Neural Information Processing Systems 28 (NIPS 2015)*.

A Detailed Misclassification Examples

B Classification Reports and Confusion Matrices

C Additional Figures